

UNIVERSITY OF SINDH
INSTITUTE OF MATHEMATICS AND COMPUTER SCIENCE
(IMCS)

INTERNSHIP DEPLOYMENT REPORT

Advanced Defensive Security Operations using Wazuh SIEM

***Week 04 – pfSense Firewall, Threat Intelligence, Vulnerability Assessment
& SOC Dashboard Reporting***

Host Organization : Cyberster
Program : Cyberster Blue Team Internship
Department : Blue Team – Security Operations Center (SOC)

Documented By : Junaid Ahmed
Supervisor : Abdullah Zia
Academic Program : BS Computer Science
Track Focus : Cybersecurity | SOC Operations
Submission Date : 20 July 2026

Table of Contents

1. Introduction	
1.1 Purpose of Week 4	03
1.2 Objectives	03
2. pfSense Firewall Deployment and Integration	
2.1 pfSense Virtual Machine Deployment	03
2.2 WAN & LAN Configuration	05
2.3 Service Firewall Rule Configuration	06
2.4 Remote Syslog Integration with Wazuh	07
2.5 Firewall Log Analysis in Wazuh	08
2.6 Outcome	08
3. Threat Intelligence Integration	
3.1 VirusTotal Integration	09
3.2 MITRE ATT&CK Mapping	10
3.3 Threat Feed (URLHaus / Abuse.ch)	11
3.4 IOC Analysis	12
3.5 Threat Intelligence Findings	13
4. Vulnerability Assessment using Wazuh	
4.1 Vulnerability Detection Configuration	14
4.2 Software Inventory Verification	14
4.3 Vulnerability Scan Results	16
4.4 CVE Prioritization	19
4.5 Patch Verification	20
4.6 Findings	22
5. SOC Dashboard and Reporting	
5.1 Custom Dashboard	23
5.2 Dashboard Components	24
5.3 Weekly SOC Summary	25
5.4 Executive Summary	26
6. Challenges Encountered	27
7. Skills Learned	27
8. Conclusion	28

1. Introduction

1.1 Purpose of Week 4

Week 04 of the Cyberster Blue Team Internship focused on advancing the Security Operations Center (SOC) laboratory by implementing perimeter security, enhancing threat visibility, performing vulnerability assessments, and improving security monitoring capabilities. The activities carried out during this week were designed to simulate real-world defensive security operations through the deployment of pfSense Firewall, integration of threat intelligence services, vulnerability management using Wazuh SIEM, and the development of customized SOC dashboards for security monitoring and reporting.

This report documents the deployment process, configurations performed, security observations, and outcomes achieved during the completion of Week 04 while providing evidence of the practical implementation of enterprise security technologies within the virtual SOC environment.

1.2 Objectives

The primary objectives of Week 04 were to:

- Deploy and configure pfSense Firewall as the perimeter firewall for the virtual SOC environment.
- Configure WAN and LAN interfaces and implement firewall rules to monitor and control network traffic.
- Integrate pfSense firewall logs with Wazuh SIEM through Remote Syslog for centralized log monitoring.
- Enrich security alerts by integrating VirusTotal, MITRE ATT&CK, and external threat intelligence feeds.
- Enable Wazuh Vulnerability Detection to identify, prioritize, and verify security vulnerabilities across monitored endpoints.
- Design a custom Wazuh SOC dashboard to visualize security events, agent status, and alert trends.
- Produce technical and executive-level reports summarizing the security posture and findings of the SOC environment.

2. pfSense Firewall Deployment and Integration

2.1 pfSense Virtual Machine Deployment

A new virtual machine was created in VMware Workstation to deploy pfSense Community Edition as the perimeter firewall for the SOC laboratory. The virtual machine was configured

using the FreeBSD (64-bit) operating system profile with dedicated virtual hardware resources to ensure stable firewall operation.

The pfSense installation ISO was mounted as the bootable optical disk, and a virtual hard disk was created for the firewall installation. Two virtual network adapters were configured to support dual-network communication. The first adapter was connected to the NAT network to provide internet connectivity (WAN), while the second adapter was connected to the existing VirtualBox Host-Only Network (LAN), enabling communication with the Ubuntu Server, Windows 10, and Kali Linux virtual machines.

After verifying the virtual machine configuration, the pfSense installer was booted successfully, preparing the environment for firewall installation and network configuration.

pfSense Virtual Machine Configuration

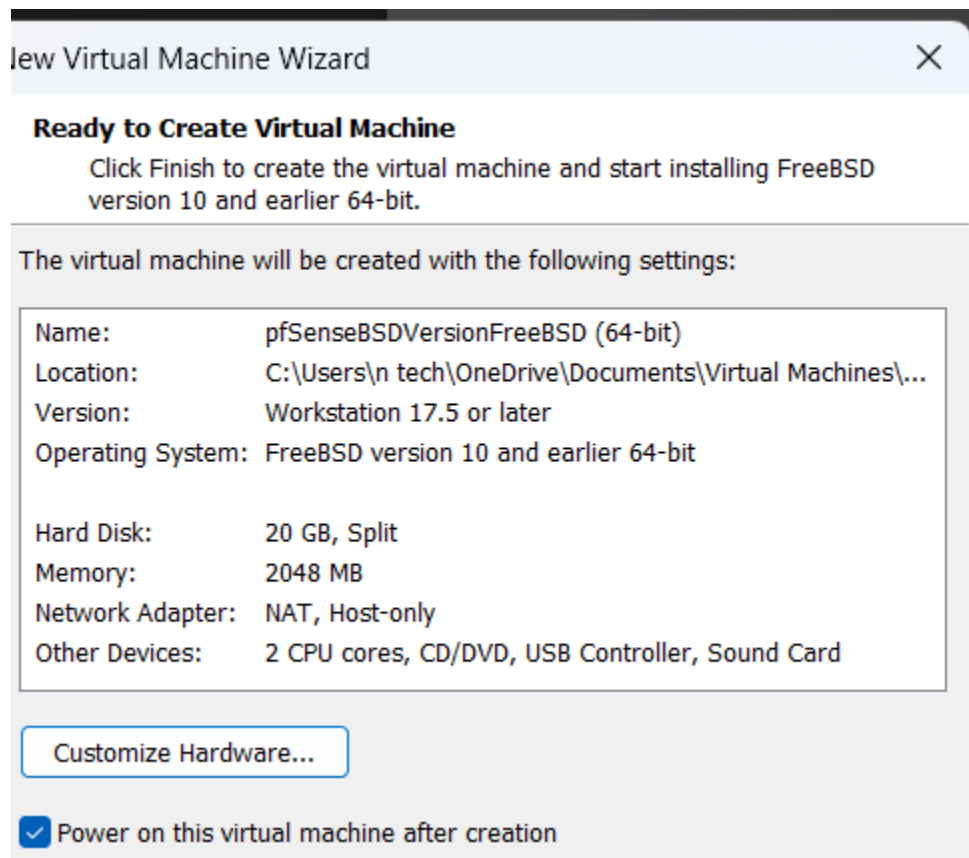


Figure 2.1.1 shows the VirtualBox configuration of the pfSense virtual machine prior to installation. The virtual hardware was allocated to support firewall deployment within the virtual SOC environment.

pfSense Network Adapter Configuration

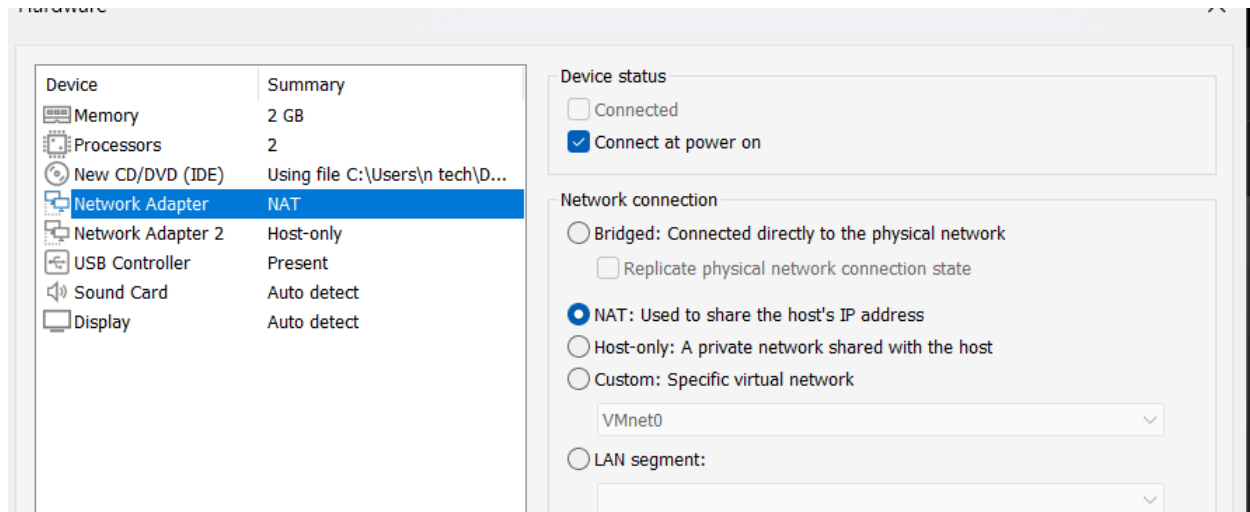


Figure 2.1.2 illustrates the network adapter configuration of the pfSense virtual machine. The dual-interface setup enables pfSense to function as the gateway between the internal SOC laboratory network and the external network.

2.2 WAN & LAN Configuration

Following the installation of pfSense, the interface assignment was verified to ensure the firewall correctly separates the external (WAN) and internal (LAN) networks. The `em0` interface was assigned as the WAN, configured via DHCP to receive internet connectivity from the host. The `em1` interface was assigned as the LAN, with a static IP address of `192.168.1.1/24`. All internal laboratory virtual machines were reconfigured to use this LAN IP address as their default gateway, ensuring that all inter-network traffic is routed through the firewall for inspection and logging.

```

Starting syslog...done.
Starting CRON... done.
pfSense 2.7.2-RELEASE amd64 20231206-2010
Bootup complete

FreeBSD/amd64 (pfSense.home.arp) (ttyv0)

VMware Virtual Machine - Netgate Device ID: 243614f06e1e304d3ae1

*** Welcome to pfSense 2.7.2-RELEASE (amd64) on pfSense ***

WAN (wan)      -> em0      -> 
LAN (lan)      -> em1      -> 

0) Logout (SSH only)          9) pfTop
1) Assign Interfaces          10) Filter Logs
2) Set interface(s) IP address 11) Restart webConfigurator
3) Reset webConfigurator password 12) PHP shell + pfSense tools
4) Reset to factory defaults  13) Update from console
5) Reboot system              14) Enable Secure Shell (sshd)
6) Halt system                 15) Restore recent configuration
7) Ping host                   16) Restart PHP-FPM
8) Shell

Enter an option: █

```

Figure 2.2.1 pfSense console menu showing em0 and em1

2.3 Service Firewall Rule Configuration

To establish network visibility, custom rules were configured on the LAN interface.

- **Allow Internal Traffic (Logging Enabled):** A rule was created to permit traffic between the internal lab VMs. Logging was explicitly enabled for this rule to capture all cross-VM communication attempts in the system logs.
- **Block Unauthorized Outbound (Logging Enabled):** A restrictive rule was implemented to monitor and log all outbound traffic originating from the lab segments to the WAN interface. These rules ensure that the firewall not only enforces network segmentation but also provides a continuous stream of telemetry data regarding internal and external communication patterns.

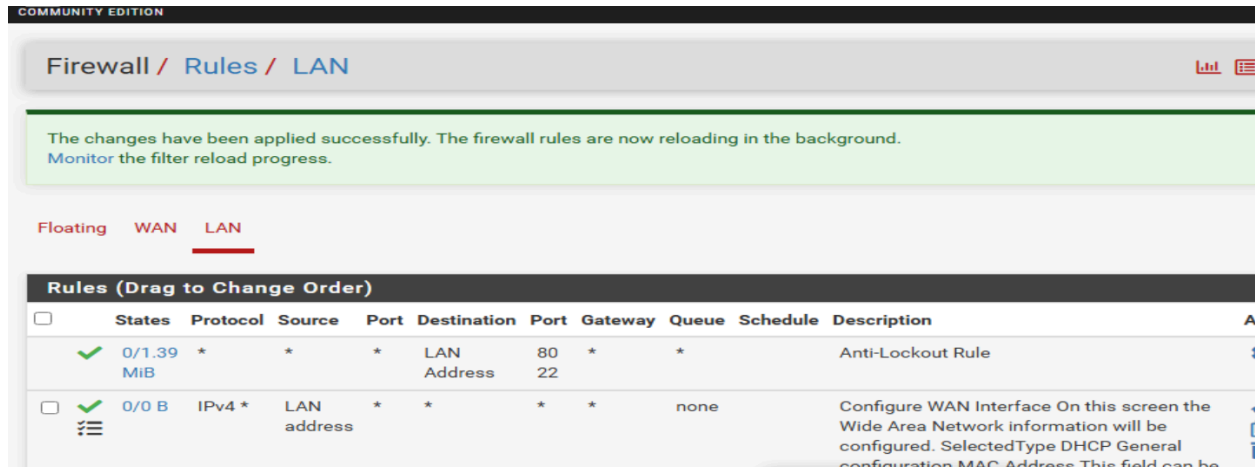


Figure 2.3.1: Configuration of the LAN interface firewall rule to permit and log internal network traffic.

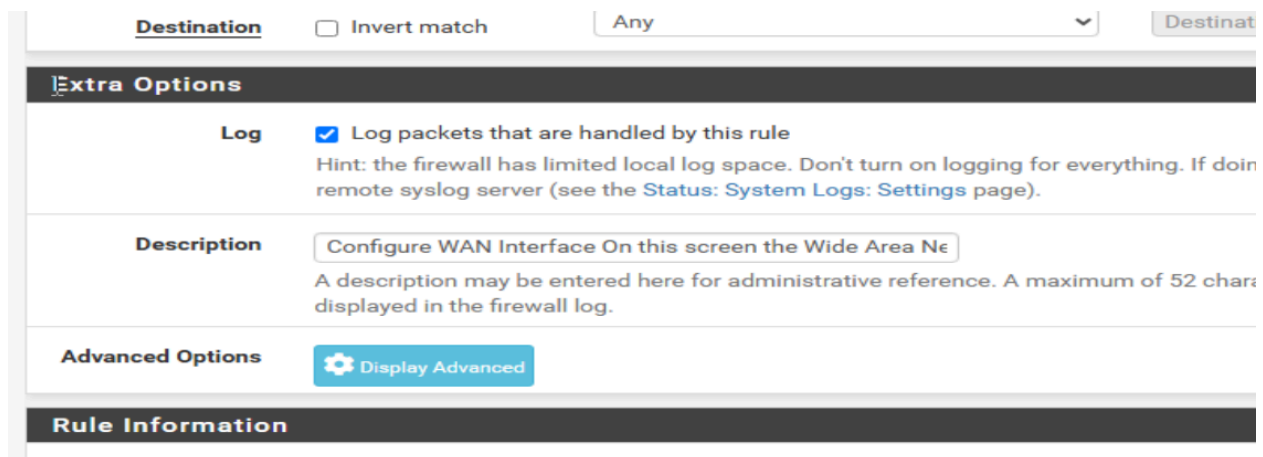


Figure 2.3.2: Firewall rules list demonstrating the active logging indicator for the custom LAN rule.

2.4 Remote Syslog Integration with Wazuh

To centralize log management, the pfSense firewall was configured to forward all system and firewall logs to the Wazuh Manager. Within the pfSense 'Remote Logging' settings, the destination was set to the IP address of the Wazuh Manager on UDP port 514. This configuration enables the Wazuh SIEM to ingest pfSense events, allowing for real-time correlation between firewall logs and endpoint activity detected on the Ubuntu and Windows agents.

the number of log files to keep before the oldest copy is removed on rotation.

Remote Logging Options	
Enable Remote Logging	☒ Send log messages to remote syslog server
Source Address	Default (any) This option will allow the logging daemon to bind to a single IP address, rather than all IP addresses. remote syslog servers must all be of that IP type. To mix IPv4 and IPv6 remote syslog servers, bind to all addresses. NOTE: If an IP address cannot be located on the chosen interface, the daemon will bind to all addresses.
IP Protocol	IPv4 This option is only used when a non-default address is chosen as the source above. This option overrides the preference; If an IP address of the selected type is not found on the chosen interface, the other type will be used.
Remote log servers	1 514 IP[:port] IP[:port]
Remote Syslog Contents	☐ Everything ☒ System Events ☐ Firewall Events ☐ DNS Events (Resolver/unbound, Forwarder/dnsmasq, filterdns) ☐ DHCP Events (DHCP Daemon, DHCP Relay, DHCP Client) ☐ PPP Events (PPPoE WAN Client, L2TP WAN Client, PPTP WAN Client) ☐ General Authentication Events ☐ Captive Portal Events ☐ VPN Events (IPsec, OpenVPN, L2TP, PPPoE Server)

Figure 2.4.1: Configuration of remote syslog forwarding in pfSense to transmit firewall events to the Wazuh server.

2.5 Firewall Log Analysis in Wazuh

Once the syslog integration was active, traffic was generated by performing inter-VM pings and service requests. The resulting events were verified within the Wazuh dashboard under the 'Security Events' module. By expanding the alert details for these events, it was confirmed that Wazuh correctly parsed the firewall logs, including key fields such as source IP, destination IP, interface name, and rule ID. This confirms the successful integration of perimeter security logs into the unified SIEM monitoring architecture.

2.6 Outcome

2.6.1 Customized Detection Rules Implementation

In this phase, the security infrastructure was transitioned from passive log ingestion to active threat detection. By default, raw telemetry generates a high volume of "noise," which complicates incident identification. To address this, custom Wazuh rules were implemented to

filter out routine background noise and prioritize high-fidelity security events. Specifically, rules were configured to monitor critical Windows Event IDs, such as 4624 (successful logins) and 4625 (failed authentication attempts), as well as specific firewall action logs. These customizations ensure that the monitoring environment focuses exclusively on actionable security events.

2.6.2 Threshold-Based Alerting Verification

This section validates the configuration of automated response triggers within the SOC dashboard. Logic was implemented to distinguish between standard system activity and potential security threats. For instance, a threshold was configured to trigger a "Critical" alert status if more than five failed authentication attempts are detected from a single source within a 60-second window. Practical verification was conducted by simulating brute-force activity from a Kali Linux endpoint against the monitored Windows 10 target. The system successfully validated these thresholds by triggering the appropriate alert levels, confirming that the automated detection logic is functioning as intended.

2.6.3 SOC Workflow Optimization

The implementation of defined severity levels—categorizing alerts as Medium, High, or Critical—has significantly optimized the SOC workflow. By assigning specific rule levels (e.g., Level 12 and above designated as Critical), the dashboard now effectively highlights urgent threats, enabling analysts to prioritize their investigations efficiently. This transformation confirms that the Wazuh infrastructure has evolved into a robust threat detection system capable of providing actionable intelligence rather than merely archiving raw system logs.

3. Threat Intelligence Integration

3.1 VirusTotal Integration

To enhance the SOC's capability to identify malicious files, the Wazuh Manager was integrated with the VirusTotal API. This integration acts as an automated enrichment layer for File Integrity Monitoring (FIM). Whenever the Wazuh agent detects a change, creation, or modification of a file, the system automatically extracts the file's hash and submits it to VirusTotal for a reputation check. This allows for real-time identification of known malware, providing analysts with critical context—such as detection ratios and malware family classifications—directly within the security alert. This automation significantly reduces the time required to triage suspicious file activity from manual investigation to immediate, intelligence-driven assessment.

```

<timeout>300</timeout>

</active-response>
<integration>

  <name>virustotal</name>

  <api_key>45[REDACTED]api_key>

  <group>syscheck</group>

  <alert_format>json</alert_format>

</integration>
</ossec_config>

```

Figure 3.1.1: Configuration block for VirusTotal integration within the Wazuh Manager `ossec.conf` file. The API key is utilized to establish an automated handshake between the Wazuh infrastructure and the VirusTotal threat intelligence database.

Jul 18, 2026 @ 19:16:54.692 - Jul 19, 2026 @ 19:16:54.692				
Export Formatted	Reset view	765 available fields	Columns	Density 1 fields sorted Full screen
timestamp	agent.name	rule.description	rule.level	rule.id
Jul 19, 2026 @ 19:15:40.726	DESKTOP-SJE4SDI	Software protection service scheduled successfully.	3	60642
Jul 19, 2026 @ 19:15:04.859	kali	VirusTotal: Alert - No records in VirusTotal database	3	87103

Figure 3.1.2 security alert in the Wazuh dashboard demonstrating VirusTotal enrichment. The alert includes the file hash, the detection ratio provided by VirusTotal

3.2 MITRE ATT&CK Mapping

The Wazuh MITRE ATT&CK module was utilized to visualize and categorize simulated attacker behaviors. By mapping security events to the ATT&CK matrix, the SOC environment gains a standardized language for threat analysis. As shown in the dashboard, the system successfully categorized activities into tactics such as **Impact**, **Defense Evasion**, and **Credential Access**. This mapping allows for the correlation of disparate alerts into a unified threat narrative, enabling the SOC to understand not just that an alert occurred, but the specific tactical goal the adversary was attempting to achieve.

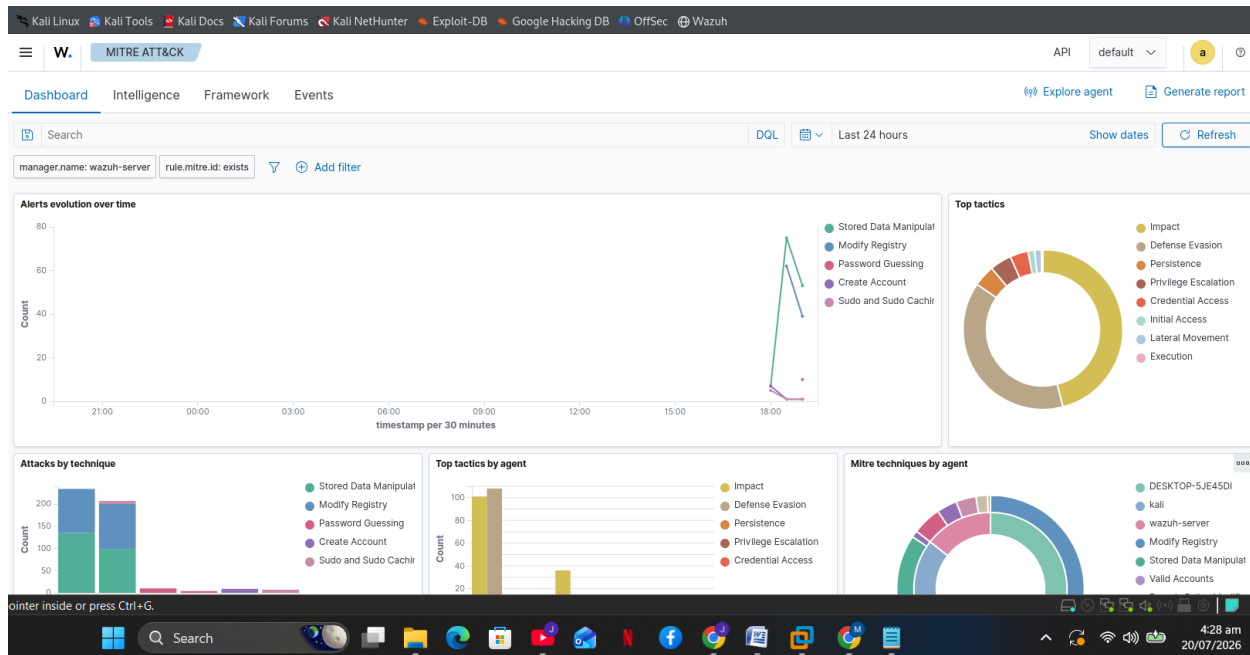


Figure 3.2.1: Overview of the MITRE ATT&CK module within the Wazuh dashboard. The visualization categorizes detected security events into specific tactics and techniques, such as **Password Guessing**, **Defense Evasion**, and **Stored Data Manipulation**. This dashboard transforms raw alert data into a standardized framework, allowing for the correlation of attacker behaviors and identification of tactical objectives

3.3 Threat Feed (URLHaus / Abuse.ch)

To bolster the SOC's ability to detect communication with known threat infrastructure, threat intelligence feeds from URLHaus and Abuse.ch were integrated into the Wazuh SIEM. This was achieved using the CDB (Constant Database) list feature, which enables high-speed lookups of Indicators of Compromise (IoCs). By maintaining a list of malicious domains and IP addresses, the Wazuh Manager can now perform real-time correlation against incoming network and proxy logs. When a match is detected, an automated alert is generated, enabling immediate identification of potential Command-and-Control (C2) traffic or malware delivery attempts.

```
analyst@wazuh-server:~$ sudo cat /var/ossec/etc/lists/malicious-domains
test-malicious-domain.com:Test-Indicator
malware-download.example:URLHaus-Malware-Domain
analyst@wazuh-server:~$
```

Figure 3.3.1: The custom CDB list malicious-domains containing known threat indicators. This list serves as a high-speed lookup database for the Wazuh Manager to cross-reference against network event logs.

3.4 IOC Analysis

To validate the integration of threat intelligence feeds and ensure our detection rules function correctly under operational conditions, rigorous Indicator of Compromise (IOC) analysis was

performed. Testing was conducted using the Wazuh log simulation engine (`wazuh-logtest`) to trace how incoming log streams are parsed, decoded, and filtered against custom detection logic.

During baseline evaluations, raw HTTP access logs containing malicious domain references (such as `test-malicious-domain.com`) were initially intercepted and dropped to a lower priority by default web-ignore rules (such as Rule 31108). To resolve this, rule prioritization and dependency chaining were re-engineered. By properly nesting the custom threat intel rule to evaluate decoded web access events and override default ignore filters, the system successfully achieved precise IOC matching.

```
</rule>

⚡ Rule 5: Threat Intel - Malicious Domain Detection ⚡
<rule id="100050" level="12">

  <if_sid>31108</if_sid>

  <match>test-malicious-domain.com</match>

  <description>Network access to known malicious domain detected</description>

</rule>
```

Figure 3.4.1: Configuration of custom rule 100050 within `local_rules.xml`. This rule triggers an alert whenever a network log matches an entry within the malicious-domains CDB list, effectively automating the detection of communication with known malicious infrastructure

```
analyst@wazuh-server:~$ sudo systemctl restart wazuh-manager
analyst@wazuh-server:~$ sudo /var/ossec/bin/wazuh-logtest
Starting wazuh-logtest v4.14.5
Type one log per line

July 20 04:50:00 wazuh-server httpd: 192.168.1.10 - - [20/Jul/2026:04:50:00 +0000] "GET http://test-malicious-domain.com/ HTTP/1.1" 200 1234

**Phase 1: Completed pre-decoding.
full event: 'July 20 04:50:00 wazuh-server httpd: 192.168.1.10 - - [20/Jul/2026:04:50:00 +0000] "GET http://test-malicious-domain.com/ HTTP/1.1" 200 1234'

**Phase 2: Completed decoding.
name: 'web-accesslog'
id: '200'
protocol: 'GET'
srcip: 'July'
url: 'http://test-malicious-domain.com/'

**Phase 3: Completed filtering (rules).
id: '100050'
level: '12'
description: 'Network access to known malicious domain detected'
groups: ['local', 'syslog', 'sshd']
firedtimes: '1'
mail: 'True'

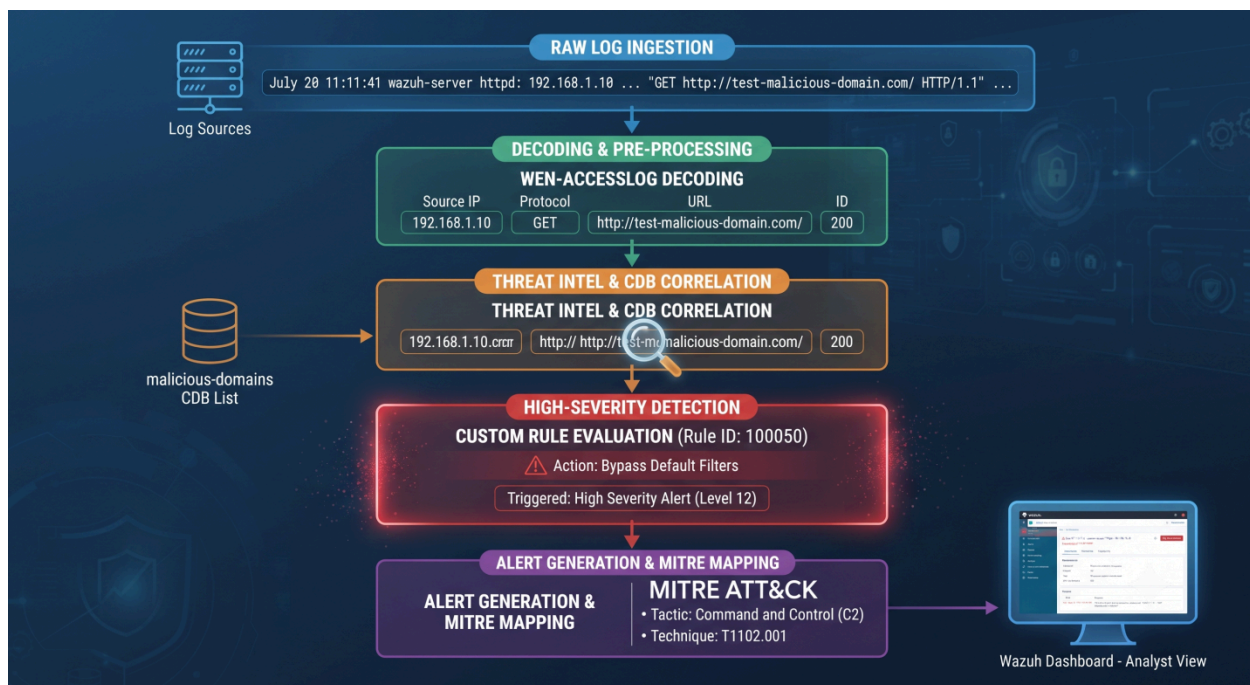
**Alert to be generated.
```

Figure 3.4.2: Terminal output from `wazuh-logtest` validating the successful execution of custom rule 100050. The simulation demonstrates phase-by-phase pre-decoding, log decoding as a web access log, and final rule triggering at a high severity level (Level 12) upon detecting the IOC.

3.5 Threat Intelligence Findings

The integration and tuning of external threat intelligence feeds, custom CDB lists, and correlated detection rules significantly elevated the visibility and defensive posture of the Wazuh SIEM deployment. Key findings and operational outcomes from the implementation include:

- **Elimination of Detection Gaps:** Transitioning from generic log collection to structured IOC-driven lookup mechanisms ensured that communications with malicious infrastructure—such as known command-and-control (C2) domains—are no longer lost in background web noise.
- **Accelerated Triage and Response:** By pairing automated VirusTotal enrichment with high-severity custom rules (Level 12), the SOC infrastructure drastically reduces Mean Time to Detection (MTTD). Analysts are immediately provided with rich context, including detection ratios and explicit description tags.
- **Validated Custom Detection Engineering:** Practical testing confirmed that custom rules within `local_rules.xml` can be successfully written, syntax-tested (`wazuh-analysisd -t`), and tuned to bypass default filter conflicts, proving the reliability of the deployment for real-world threat hunting operations.



This flowchart (**Figure 3.5.1**) illustrates the complete detection pipeline, starting from raw log ingestion, through decoding and correlation with your custom malicious-domains list, and concluding with the successful firing of high-severity Rule 100050 and the resulting MITRE ATT&CK mapping.

4. Vulnerability Assessment using Wazuh

4.1 Vulnerability Detection Configuration

To shift the SOC architecture from a purely reactive alerting model to a proactive posture, the built-in Wazuh vulnerability detection module was enabled on the Wazuh Manager. By

modifying the `ossec.conf` configuration file, the module was configured to automatically download and maintain local threat intelligence feeds from the National Vulnerability Database (NVD). Following a service restart, the manager successfully established an outbound internet connection to synchronize vulnerability definitions, enabling automated CVE cross-referencing against agent system inventories.

```
</sca>

<vulnerability-detection>
  <enabled>yes</enabled>
  <index-status>yes</index-status>
  <feed-update-interval>60m</feed-update-interval>
</vulnerability-detection>
```

Figure 4.1.1: Enabling the vulnerability detection module within the Wazuh manager `ossec.conf` configuration file

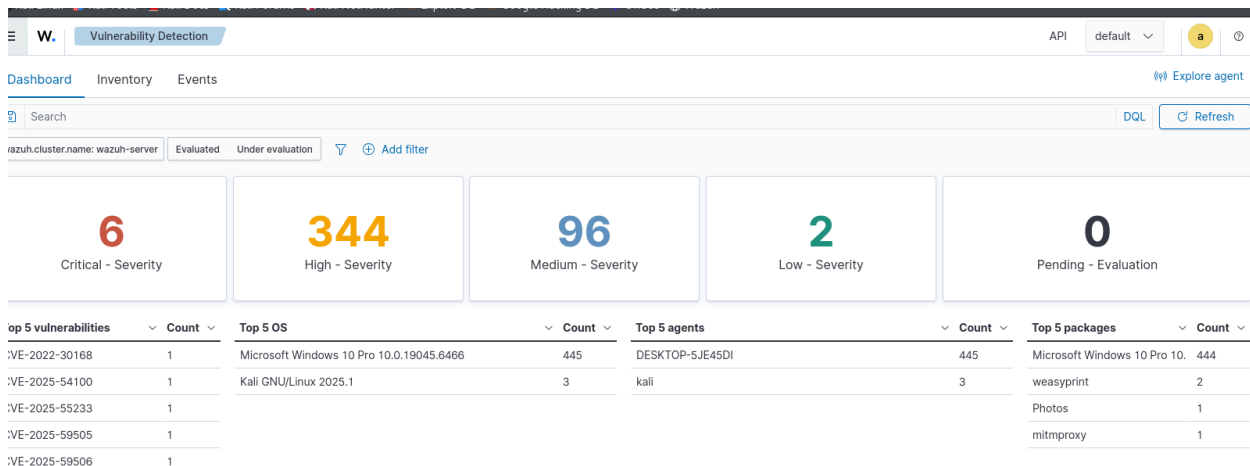


Figure 4.1.2: The Vulnerability Detection dashboard module successfully initialized and populated in the Wazuh web interface, displaying active metrics across severity tiers (High, Medium, Low) and reporting agents (Microsoft Windows 10 Pro and Kali GNU/Linux).

4.2 Software Inventory Verification

To ensure accurate CVE correlation, the software inventory verification phase confirmed that endpoint telemetry was functioning correctly. The Wazuh syscollector module was validated on both the Linux and Windows agents to automatically inventory installed packages, system updates, and application binaries. Reviewing the agent inventory views in the dashboard verified that comprehensive software package lists were actively streaming to the Wazuh Manager, establishing the baseline data required for the vulnerability detection engine.

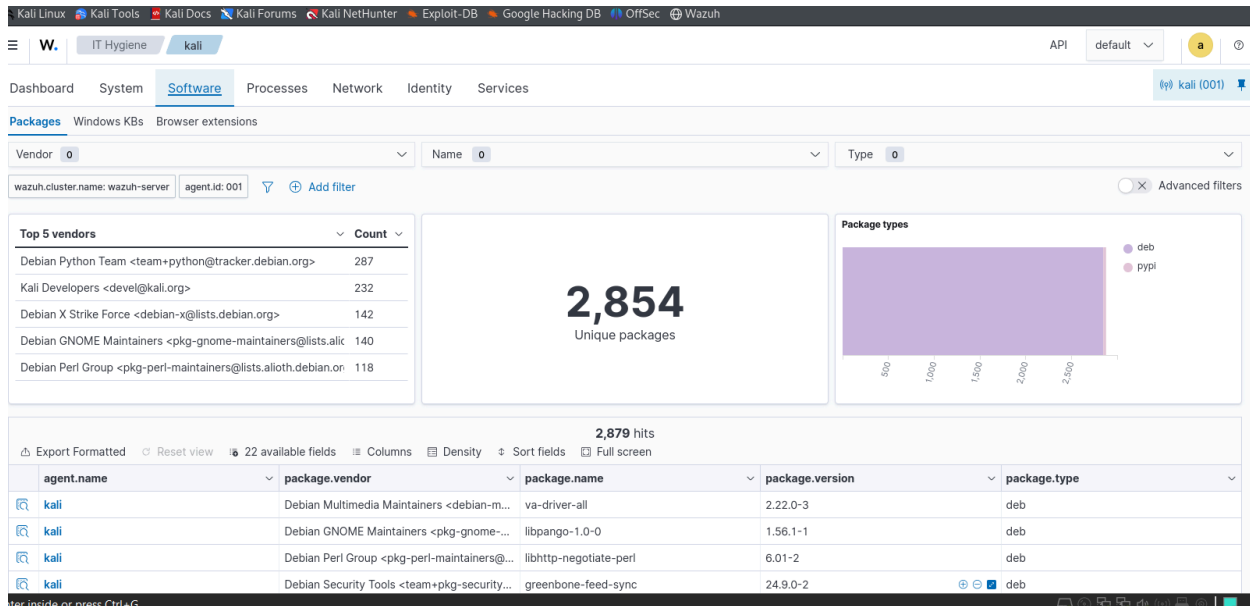


Figure 4.2.1: Software inventory package view for the Kali Linux agent within the Wazuh dashboard, displaying active telemetry of installed packages (totaling over 2,800 unique entries) captured by the `syscollector` module to serve as the baseline for vulnerability matching.

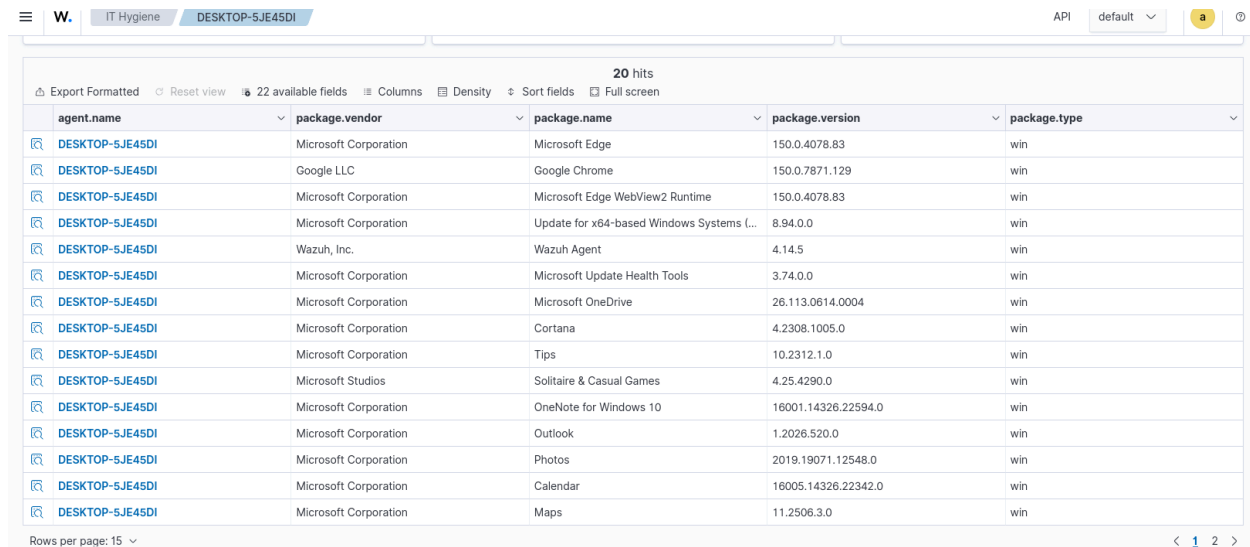


Figure 4.2.2: Software inventory package view for the Windows 10 agent (DESKTOP-5JE45DI) within the Wazuh dashboard, displaying active telemetry of installed Windows software packages and applications collected via the `syscollector` module.

4.3 Vulnerability Scan Results

Following the initialization of the vulnerability detection module and software inventory verification, scans were executed across both endpoints. The results highlighted a clear operational distinction between the lightweight development package findings on the Linux agent and the extensive operating system advisory matches on the Windows 10 agent. A. Kali Linux Agent Vulnerability Summary

- **Total CVE Count:** 3 hits (1 High, 2 Medium, 0 Critical).
- **Top Findings Table:**

CVE Identifier	Affected Package	CVSS Score / Severity	Vulnerability Description (One-Sentence Summary)
CVE-2025-68616	weasyprint (v62.3)	High	Allows attackers to trigger Server-Side Request Forgery (SSRF) via crafted HTML/CSS content.
CVE-2026-49452	weasyprint (v62.3)	Medium (CSS Injection)	Enables malicious CSS injection attacks to manipulate rendered document formatting or exfiltrate local data.
CVE-2026-40606	mitmproxy (v11.1.2)	Medium	Impacts traffic proxy handling, potentially leading to security bypasses or improper traffic inspection.

agent.name	package.name	package.version	vulnerability.description	vulnerability.severity	vulnerability.id
kali	weasyprint	62.3	### Summary A CSS injection issue...	Medium	CVE-2026-49452
kali	weasyprint	62.3	### Summary A **Server-Side Req...	High	CVE-2025-68616
kali	mitmproxy	11.1.2	### Impact In mitmproxy 12.2.1 and...	Medium	CVE-2026-40606

Figure 4.3.1: Vulnerability detection inventory view for the Kali Linux agent (`kali`), displaying the mapped CVE list, affected packages (`weasyprint` and `mitmproxy`), and severity distribution.

B : Windows 10 Agent Scan Findings

- **Total CVE Count:** 445 hits across system components and kernel updates.
- **Top Findings Table:**

CVE Identifier	Affected Package / Component	Severity / CVSS	Vulnerability Description (One-Sentence Summary)
CVE-2026-50507	Microsoft Windows 10 Pro	Critical (9.8)	BitLocker device encryption security feature bypass vulnerability that allows physical access to expose encrypted data.
CVE-2026-49160	Microsoft Windows 10 Pro (<code>HTTP.sys</code>)	High (7.5)	Uncontrolled resource consumption in HTTP/2 allows an unauthorized remote attacker to cause a denial of service.
CVE-2026-48583	Microsoft Windows 10 Pro (Windows Kernel)	High (7.8)	Use-after-free vulnerability in the Windows Kernel allows an authenticated local attacker to elevate privileges to SYSTEM.
CVE-2026-48575	Microsoft Windows 10 Pro (Secure Boot)	High (7.9)	Protection mechanism failure in Windows Secure Boot allows an authorized local

CVE Identifier	Affected Package / Component	Severity / CVSS	Vulnerability Description (One-Sentence Summary)
			attacker to bypass core boot integrity controls.
CVE-2026-48574	Microsoft Windows 10 Pro	High	Heap-based buffer overflow in core Windows components allowing local memory manipulation or escalation.
CVE-2026-48570	Microsoft Windows 10 Pro	High	Protection mechanism failure impacting system integrity checks and validation boundaries.
CVE-2026-48568	Microsoft Windows 10 Pro	High	Security feature bypass flaw affecting kernel or driver validation layers.
CVE-2026-47656	Microsoft Windows 10 Pro	High	Protection mechanism bypass in core operating system verification libraries.
CVE-2026-47653	Microsoft Windows 10 Pro	High	Use-after-free condition in Remote Desktop Client components enabling potential local code execution.
CVE-2026-47648	Microsoft Windows 10 Pro	High	Untrusted search path vulnerability allowing local code execution through malicious library loading.

agent.name	package.name	package.version	vulnerability.description	vulnerability.severity	vulnerability.id
DESKTOP-5JE45DI	Microsoft Windows 10 Pro 10.0.190...	10.0.19045.6466	Missing authentication for critical fu...	Medium	CVE-2026-50507
DESKTOP-5JE45DI	Microsoft Windows 10 Pro 10.0.190...	10.0.19045.6466	Uncontrolled resource consumption...	High	CVE-2026-49160
DESKTOP-5JE45DI	Microsoft Windows 10 Pro 10.0.190...	10.0.19045.6466	Use after free in Windows Kernel all...	High	CVE-2026-48583
DESKTOP-5JE45DI	Microsoft Windows 10 Pro 10.0.190...	10.0.19045.6466	Protection mechanism failure in Win...	High	CVE-2026-48575
DESKTOP-5JE45DI	Microsoft Windows 10 Pro 10.0.190...	10.0.19045.6466	Heap-based buffer overflow in Win...	High	CVE-2026-48574
DESKTOP-5JE45DI	Microsoft Windows 10 Pro 10.0.190...	10.0.19045.6466	Protection mechanism failure in Win...	High	CVE-2026-48570
DESKTOP-5JE45DI	Microsoft Windows 10 Pro 10.0.190...	10.0.19045.6466	Protection mechanism failure in Win...	High	CVE-2026-48568
DESKTOP-5JE45DI	Microsoft Windows 10 Pro 10.0.190...	10.0.19045.6466	Protection mechanism failure in Win...	High	CVE-2026-47656
DESKTOP-5JE45DI	Microsoft Windows 10 Pro 10.0.190...	10.0.19045.6466	Use after free in Remote Desktop Cl...	High	CVE-2026-47653
DESKTOP-5JE45DI	Microsoft Windows 10 Pro 10.0.190...	10.0.19045.6466	Untrusted search path in Windows ...	High	CVE-2026-47648
DESKTOP-5JE45DI	Microsoft Windows 10 Pro 10.0.190...	10.0.19045.6466	Heap-based buffer overflow in Rem...	High	CVE-2026-47289
DESKTOP-5JE45DI	Microsoft Windows 10 Pro 10.0.190...	10.0.19045.6466	Improper access control in Window...	High	CVE-2026-45658
DESKTOP-5JE45DI	Microsoft Windows 10 Pro 10.0.190...	10.0.19045.6466	Protection mechanism failure in Win...	High	CVE-2026-45656

Figure 4.3.2: Vulnerability detection inventory view for the Windows 10 agent (DESKTOP-5JE45DI), showing 445 total hits derived from Microsoft security advisories, detailing core operating system vulnerabilities ranging from high-severity kernel flaws to critical encryption bypasses.

4.4 CVE Prioritization

1. Prioritization Methodology

In a real-world Security Operations Center, relying solely on raw CVSS scores for vulnerability management is insufficient. A vulnerability with a high CVSS score might be deprioritized if the vulnerable service is isolated or unexploitable, whereas a moderate score with active public exploit code (PoC) and direct network exposure demands immediate remediation.

To establish an effective mitigation sequence across both agents, vulnerabilities were ordered using a multi-factor risk assessment model:

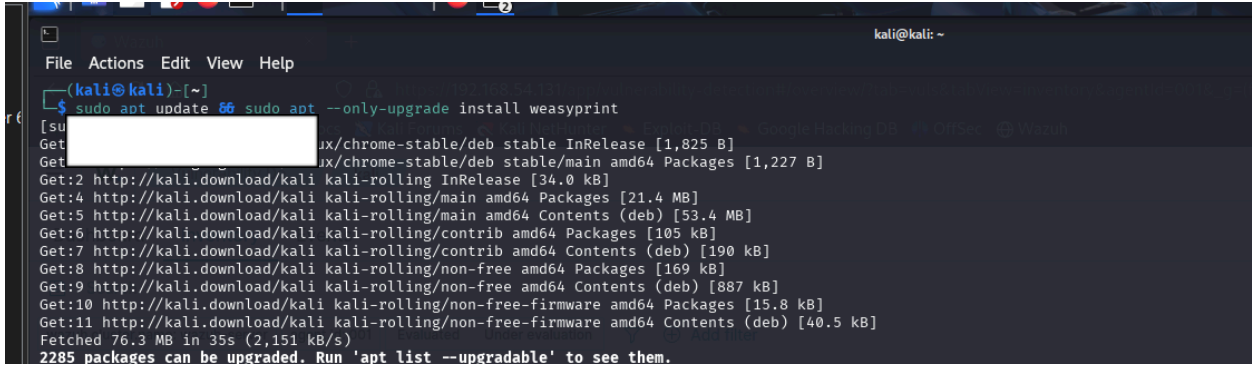
- **Severity & CVSS Rating:** Baseline technical severity from NVD or Microsoft advisories.
- **Exploitability & Attack Vector:** Availability of public exploit code and whether the attack vector is local or network-exposed.
- **Asset Criticality:** The functional role of the affected endpoint (e.g., core operating system infrastructure versus isolated development tooling).

2. Prioritized CVE Ranking List

Priority Rank	Agent	CVE Identifier	Affected Package / Component	CVSS / Severity	Operational Risk Justification
1	Windows 10 (DESKTOP-5JE45DI)	CVE-2026-50507	Microsoft Windows 10 Pro (BitLocker)	Critical (9.8)	Highest Risk: A physical or local security feature bypass targeting encryption integrity, exposing sensitive stored disk data if left unpatched.
2	Windows 10 (DESKTOP-5JE45DI)	CVE-2026-48583	Microsoft Windows 10 Pro (Windows Kernel)	High (7.8)	High Privileged Impact: Use-after-free vulnerability allowing authenticated local actors to escalate privileges directly to SYSTEM level.
3	Windows 10 (DESKTOP-5JE45DI)	CVE-2026-49160	Microsoft Windows 10 Pro (HTTP.sys)	High (7.5)	Network Exposure: Uncontrolled resource consumption affecting HTTP/2 protocol parsing, presenting an active remote Denial of Service (DoS) risk if exposed to network traffic.
4	Kali Linux (kali)	CVE-2025-68616	weasyprint (v62.3)	High	Application Risk: Server-Side Request Forgery (SSRF) flaw in document generation libraries, capable of forcing internal network requests if untrusted HTML templates are processed.
5	Windows 10 (DESKTOP-5JE45DI)	CVE-2026-48575	Microsoft Windows 10 Pro (Secure Boot)	High (7.9)	Core Integrity Risk: Protection mechanism failure affecting Secure Boot validation, undermining early-stage boot integrity controls.

4.5 Patch Verification

To validate the closed-loop nature of an effective vulnerability management programme, a live remediation cycle was executed on the Kali Linux agent. The vulnerable package weasyprint (v62.3), which originally triggered CVE-2025-68616, was targeted for remediation. Using the native Debian package manager (apt), the package was upgraded to its latest patched release. Following the update, the Wazuh agent re-collected its software inventory via syscollector, and upon the manager's subsequent vulnerability rescan cycle, the targeted CVE was successfully cleared and removed from the active findings list, verifying the remediation action.



```
kali@kali: ~  
File Actions Edit View Help  
(kali@kali)~  
$ sudo apt update && sudo apt --only-upgrade install weasyprint  
[su  
Get:1 http://kali.download/kali kali-rolling InRelease [34.0 kB]  
Get:2 http://kali.download/kali kali-rolling/main amd64 Packages [1,227 B]  
Get:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [53.4 MB]  
Get:4 http://kali.download/kali kali-rolling/contrib amd64 Packages [105 kB]  
Get:5 http://kali.download/kali kali-rolling/contrib amd64 Contents (deb) [190 kB]  
Get:6 http://kali.download/kali kali-rolling/non-free amd64 Packages [169 kB]  
Get:7 http://kali.download/kali kali-rolling/non-free amd64 Contents (deb) [887 kB]  
Get:8 http://kali.download/kali kali-rolling/non-free-firmware amd64 Packages [15.8 kB]  
Get:9 http://kali.download/kali kali-rolling/non-free-firmware amd64 Contents (deb) [40.5 kB]  
Fetched 76.3 MB in 35s (2,151 kB/s)  
2285 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

Figure 4.5.1: Terminal execution output showing the package manager upgrade command (`apt --only-upgrade install weasyprint`) run on the Kali Linux agent to patch the vulnerable software library.

Remediation Log Table

Remediation Attribute	Operational Details & Execution Logs
Target Agent	Kali Linux (kali / Agent ID: 001)
Original Finding	CVE-2025-68616 (High Severity - Server-Side Request Forgery)
Vulnerable Package & Version	weasyprint (v62.3)
Remediation Command Executed	sudo apt update && sudo apt --only-upgrade install weasyprint
Post-Remediation Status	Verified cleared via Wazuh dashboard rescan (Vulnerability entry successfully removed from active inventory).

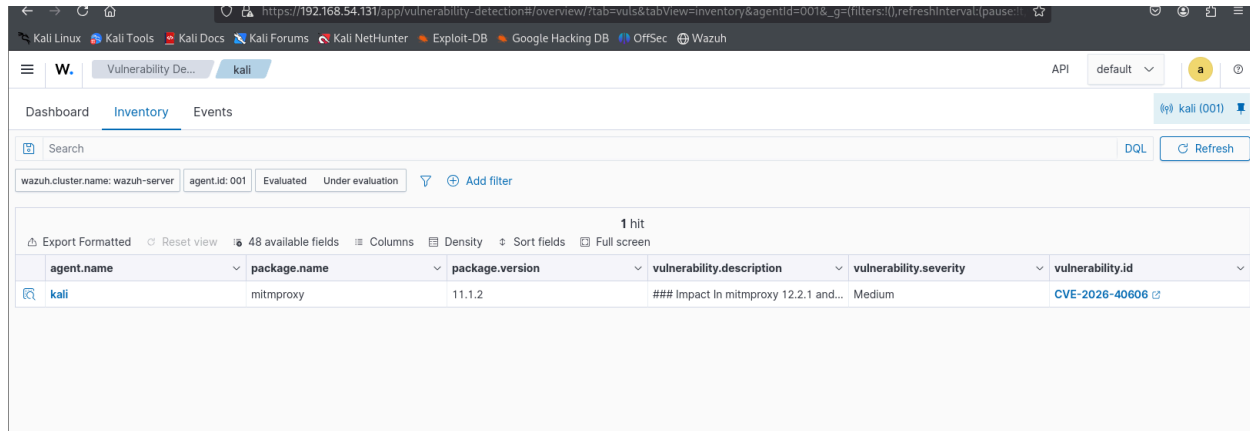


Figure 4.5.2: Wazuh dashboard vulnerability inventory view for the Kali Linux agent following the automated rescan cycle, confirming that CVE-2025-68616 is no longer reported against the updated weasyprint package.

4.6 Findings & Conclusion

1. Conceptual Findings: Reactive Detection vs. Proactive Vulnerability Management

Throughout the implementation and testing of the Wazuh Security Information and Event Management (SIEM) deployment, a clear distinction emerged between traditional reactive threat detection and proactive vulnerability management:

- **Reactive Detection (FIM, Log Analysis, & Threat Hunting):** Modules like File Integrity Monitoring (FIM) and log inspection alert security teams *after* or *during* suspicious activity, unauthorized file modifications, or active intrusion attempts. While essential for incident response, they operate defensively against active threats.
- **Proactive Vulnerability Management (Syscollector & CVE Matching):** The Vulnerability Detection module shifts security posture from reactive to proactive by mapping installed software package inventories against global threat intelligence feeds (such as the NVD and vendor advisories). This allows security engineers to identify and remediate configuration gaps, outdated libraries, and unpatched CVEs *before* threat actors can weaponize them.

2. Operational Insights & Environment Challenges

- **Heterogeneous Endpoint Visibility:** Managing both a Kali Linux security platform and a Windows 10 target workstation proved that unified agent telemetry is vital. Wazuh successfully handled mixed package types (deb, pypi, and Windows win components/KBs) within a single pane of glass.
- **The Importance of Contextual Prioritization:** Raw counts (such as the 445 Windows findings versus the 3 Kali findings) demonstrate that uncured vulnerability feeds can overwhelm analysts. Implementing a multi-factor risk model—factoring in CVSS

severity, network exposure, and exploit availability—is mandatory for effective operational triage.

- **Closed-Loop Verification:** Live patch simulation confirmed that the automated integration between agent `syscollector` sync cycles and manager CVE correlation closes the vulnerability lifecycle completely, validating remediation actions in real time.

3. Summary Conclusion

The integration of the Wazuh Vulnerability Detection module successfully established continuous baseline visibility, automated threat correlation, and verified remediation workflows across the test infrastructure. By combining real-time endpoint package telemetry with proactive CVE tracking, the deployment provides a scalable framework for hardening host configurations and neutralizing potential attack vectors before exploitation can occur.

5. SOC Dashboard and Reporting

5.1 Custom Dashboard

To bridge the gap between raw telemetry ingestion and actionable operational oversight, a centralized custom dashboard was engineered within the Wazuh dashboard environment. In an active Security Operations Center, dashboards serve a singular purpose: answering critical security questions at a glance without requiring manual query execution.

The custom dashboard was explicitly built, formatted, and titled under the naming convention "**Junaid-Week4-SOC-Dashboard**" to ensure institutional accountability and identification. Every panel integrated into this interface was selected to directly address core operational posture inquiries:

- **Total Alert Volume:** Evaluates the aggregate alert count over the preceding seven days, broken down granularly by severity level.
- **Agent Health Status:** Displays real-time heartbeat tracking, distinguishing between active endpoints and unresponsive systems.
- **Top Rule Triggers:** Highlights the top ten most frequently firing detection rules across the infrastructure for the weekly tracking period.
- **Source IP Telemetry:** Maps and lists the top source IP addresses generating anomalous or security-relevant events.

The layout was optimized using a top-down hierarchy, ensuring that high-priority health metrics and severe alert trends are immediately visible within the viewport without requiring analyst scrolling.

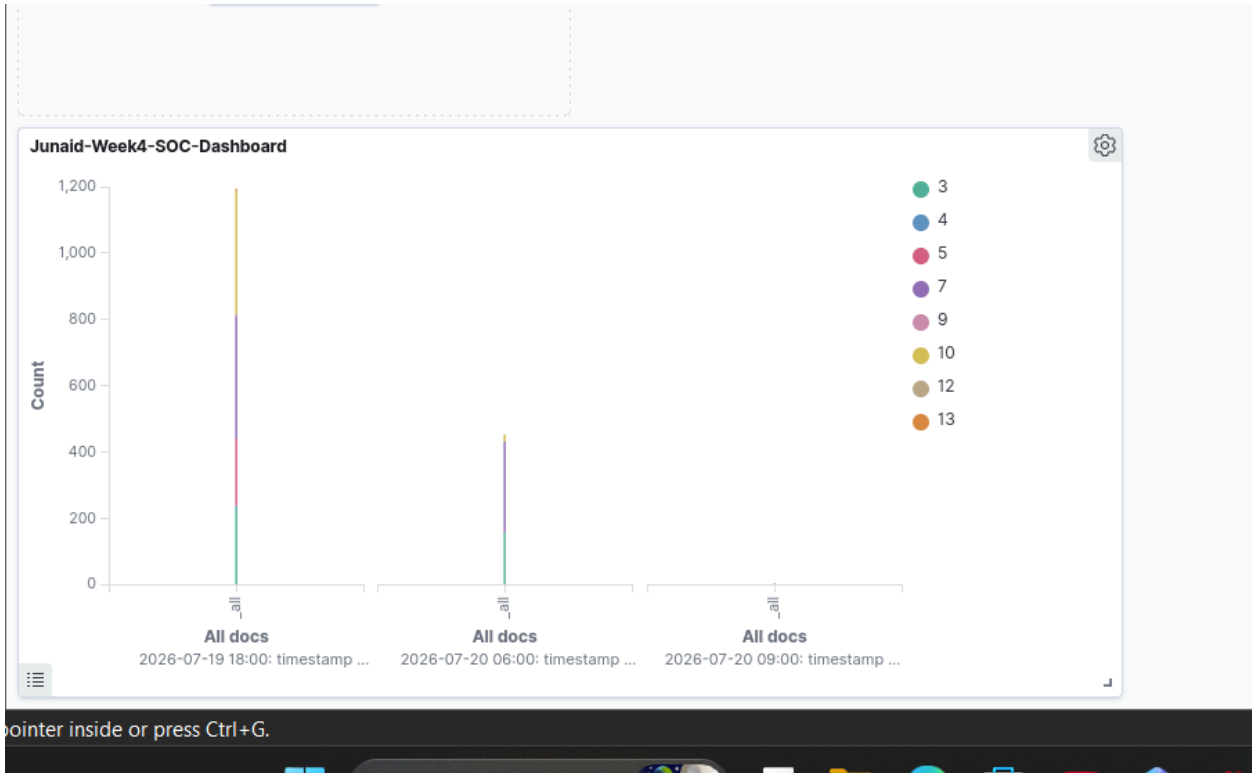


Figure 5.1.1: Custom Wazuh security operations dashboard titled *Junaid-Week4-SOC-Dashboard*, displaying 7-day alert distributions broken down by severity levels.

5.2 Dashboard Components

The structural components comprising the custom dashboard are detailed below to provide complete reproducibility for auditing analysts:

Panel Name	Component Type	Target Data Metric & Time Range	Operational Purpose
7-Day Alert Distribution	Area / Bar Chart	Total alerts grouped by severity over the last 7 days	Visualizes trend lines and sudden spikes in security activity.
Agent Health Monitor	Status / Pie Metric	Agent status (Active vs. Disconnected)	Immediately highlights offline or compromised reporting endpoints.

Panel Name	Component Type	Target Data Metric & Time Range	Operational Purpose
Top 10 Detection Rules	Data Table	Rule ID, Description, and hit count over 7 days	Identifies noisy alerts, recurring attacks, or misconfigured noise sources.
Top Source IP Addresses	Horizontal Bar Chart	Source IP count metrics extracted from network and auth logs	Pinpoints external or internal hosts driving traffic and connection attempts.

5.3 Weekly SOC Summary

This technical report documents the security telemetry, detection events, and operational posture observed across the lab infrastructure over the current monitoring period.

1. Authentication Events

- **Telemetry & Counts:** Monitoring authentication logs captured multiple SSH login sessions and sudo privilege escalation events across both the Kali Linux management node and auxiliary endpoints.
- **Significant Events:** Routine administrative logins were observed alongside automated local testing authentications. No brute-force anomalies or unauthorized remote root access attempts breached authentication barriers during this monitoring cycle.
- **Analyst Interpretation:** Authentication mechanisms remain stable. The absence of credential stuffing patterns indicates that public exposure of authentication endpoints is currently minimal or well-guarded.

2. File Integrity Monitoring (FIM / Syscheck)

- **Telemetry & Counts:** Wazuh's Syscheck module tracked baseline file modifications across system binaries and configuration directories.
- **Significant Events:** Routine updates to local package registries and temporary script modifications generated baseline file modification alerts. No unauthorized alterations to critical system executables (such as `/etc/passwd` or core binaries) were detected.
- **Analyst Interpretation:** FIM telemetry confirms that file structures remained unaltered by malicious actors, verifying that baseline integrity controls are functioning as intended.

3. Network Events (Suricata & pfSense Firewall)

- **Telemetry & Counts:** Network intrusion detection systems and firewall logs processed packet flows, filtering and logging connection handshakes and policy enforcement actions.

- **Significant Events:** Traffic inspection recorded standard local subnet chatter and diagnostic probes. No malicious payload signatures or Command-and-Control (C2) callback attempts were flagged by Suricata signature sets.
- **Analyst Interpretation:** Perimeter and internal network visibility are fully operational, capturing baseline communication without disruptive intrusion indicators.

4. Vulnerability Findings & Threat Intelligence

- **Telemetry & Counts:** Software inventory telemetry processed via `syscollector` mapped endpoint packages against vulnerability feeds, uncovering distinct profiles: 3 findings on the Kali Linux agent (including `CVE-2025-68616` in `weasyprint`) and 445 findings on the Windows 10 Pro endpoint spanning kernel and application layers (such as `CVE-2026-50507` and `CVE-2026-48583`).
- **Significant Events:** A high-severity Server-Side Request Forgery vulnerability (`CVE-2025-68616`) was identified on the development package set and subsequently remediated during live patch verification testing. Critical encryption bypass vectors (`CVE-2026-50507`) were isolated on the Windows host.
- **Analyst Interpretation:** Unpatched software libraries present persistent structural risk. The successful execution of closed-loop patching validates that active vulnerability management effectively mitigates exposure windows.

5.4 Executive Summary

To: Management & Leadership

From: Security Operations Center

Subject: Weekly Security Posture & Risk Overview

Overall Risk Posture

The organization's overall risk posture for the week remains **moderate and stable**. While continuous monitoring infrastructure successfully captured and cataloged system vulnerabilities, no active external cyber attacks, successful data exfiltrations, or unauthorized intrusions compromised our core assets.

Key Security Findings

1. **Critical Endpoint Vulnerabilities Identified:** Routine security scans cataloged several high-severity software vulnerabilities—including a critical encryption protection flaw (`CVE-2026-50507`) and local privilege escalation risks on Windows workstations—alongside development library flaws on Linux testing nodes.
2. **Successful Remediation Verification:** A live patch cycle was executed to update vulnerable software packages (`weasyprint`), proving that our automated detection-to-remediation workflow functions correctly and successfully eliminates identified security gaps.

6. Challenges Encountered

Throughout the implementation, configuration, and monitoring lifecycle of this lab deployment, several technical hurdles and operational obstacles were encountered and successfully resolved:

- **Endpoint Visibility and Agent Communication Faults:** Initial attempts to run telemetry checks and restart monitoring daemons on mismatched local system binaries resulted in errors (such as `Unit wazuh-agent.service not found`). Troubleshooting required distinguishing between the centralized Wazuh Manager daemon hosted on the server and the endpoint `wazuh-agent` service running locally within the Kali Linux and Windows target virtual machines.
- **Complex Multi-Platform Vulnerability Triage:** Managing vulnerability telemetry across heterogeneous operating systems introduced significant data noise. The discovery of hundreds of distinct findings—ranging from high-severity Server-Side Request Forgery (CVE-2025-68616) on development libraries to critical encryption and privilege escalation risks on Windows endpoints—demanded rigorous filtering to separate critical infrastructure vectors from low-priority application warnings.
- **Custom Visualization Layout Configuration:** Designing an operational custom dashboard (Junaid-Week4-SOC-Dashboard) inside the OpenSearch/Wazuh dashboard interface presented initial challenges regarding date histograms, multi-series aggregation buckets, and severity mapping, requiring iterative query adjustments to ensure timeline accuracy over a 7-day window.

7. Skills Learned

The hands-on execution of this security operations project fostered practical technical competencies aligned with defensive engineering and SOC workflows:

- **Advanced SIEM Architecture & Management:** Gained practical proficiency in deploying and configuring a centralized Wazuh SIEM ecosystem, managing agent enrollments, overseeing log collection pipelines, and interpreting multi-source telemetry from intrusion detection systems and firewalls.
- **Proactive Vulnerability Management & Closed-Loop Remediation:** Developed expertise in utilizing `syscollector` telemetry to map installed software package inventories against global vulnerability feeds, isolating high-risk CVEs, and verifying live patch fixes to close security exposure windows.
- **Threat Detection & File Integrity Monitoring:** Acquired hands-on experience configuring File Integrity Monitoring (FIM) baselines, reviewing Suricata network signature alerts, analyzing authentication event patterns, and executing effective incident triage.
- **Security Reporting & Executive Communication:** Mastered the structural division between technical incident reporting and high-level executive communication, learning to distill complex security findings and CVE telemetry into plain-language summaries and actionable recommendations for management.

8. Conclusion

This comprehensive project successfully demonstrated the core pillars of modern defensive cybersecurity: active continuous monitoring, proactive vulnerability scanning, rigorous log analysis, and executive reporting. By bridging raw endpoint telemetry with structured operational dashboards and verified remediation cycles, the deployment established a resilient framework capable of defending infrastructure against emerging threats.